



Національний технічний університет України  
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки  
Кафедра обчислювальної техніки

## Архітектура Комп'ютерів. Частина 1

### Лабораторна робота №4

«Обробка інформації в ЕОМ на програмному і мікропрограмному рівнях»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

*Давидчук А. М.*

Залікова книжка № 4106

Перевірив: *Верба О. А.*

**Тема:** «Обробка інформації в ЕОМ на програмному і мікропрограмному рівнях».

**Мета:** вивчити етапи формування системи команд процесорів, різновиди форматів команд та способів адресації операндів. Навчитися розробляти мікроалгоритми і мікропрограми виконання етапів команд з використанням мнемонічного мікроасемблера.

## Хід роботи

### Підготовка до роботи

Номер варіанту –  $4106_{10} = 0001\ 0000\ 0000\ 1010_2$ .

$$a_7 = 0, \quad a_6 = 0, \quad a_5 = 0, \quad a_4 = 1, \quad a_3 = 0, \quad a_2 = 1, \quad a_1 = 0.$$

| $a_3$ | $a_2$ | $a_1$ | Операція   |
|-------|-------|-------|------------|
| 0     | 0     | 0     | $X + Y$    |
| 0     | 0     | 1     | $X - Y$    |
| 0     | 1     | 0     | $X \& Y$   |
| 0     | 1     | 1     | $X \vee Y$ |

| $a_7$ | $a_6$ | $a_5$ | РС  | РД  |
|-------|-------|-------|-----|-----|
| 0     | 0     | 0     | 02H | 04H |
| 0     | 0     | 1     | 12H | 14H |
| 0     | 1     | 0     | 22H | 24H |
| 0     | 1     | 1     | 32H | 34H |

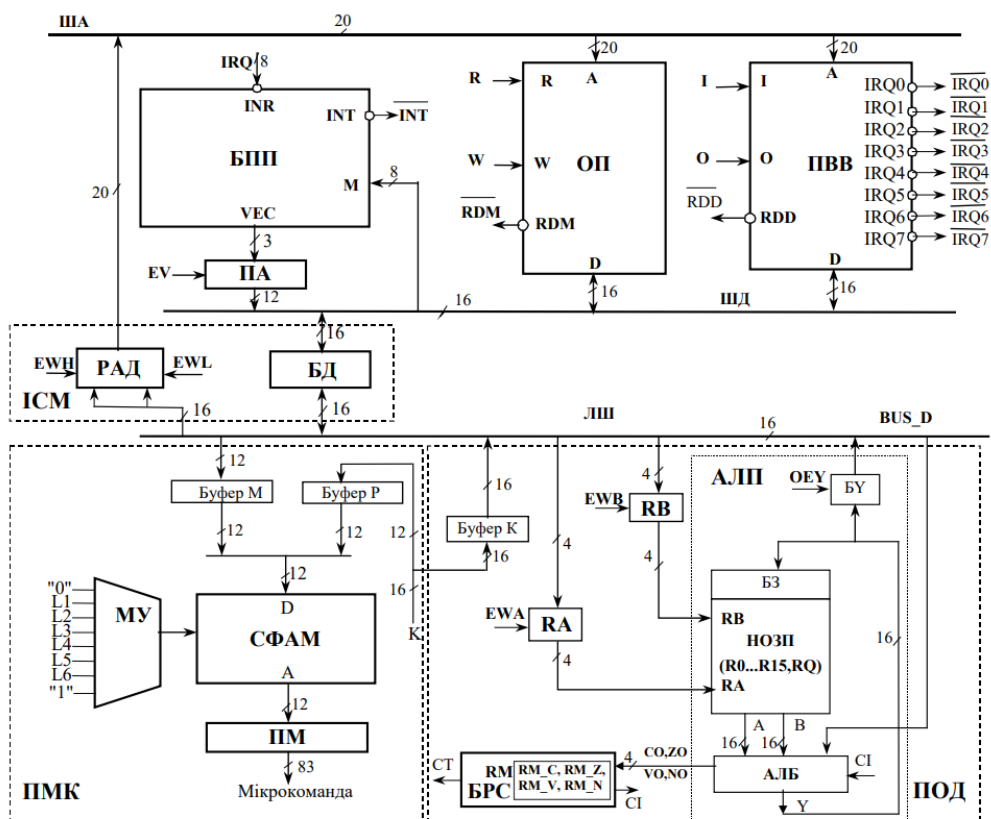
| $a_4$ | Тип адресації    |
|-------|------------------|
| 0     | Пряма            |
| 1     | Пряма регістрова |

| $a_2$ | $a_1$ | Спосіб множення |
|-------|-------|-----------------|
| 0     | 0     | 1-й             |
| 0     | 1     | 2-й             |
| 1     | 0     | 3-й             |
| 1     | 1     | 4-й             |

Для обрахунків оберемо такі значення:

$$X = 14, \quad Y = 11, \quad Z = 5.$$

## Структура ЕОМ



## Формат одноадресних команд та формат регістрів стану і даних

| Поле | Біти  | Довжина | Призначення                                    |
|------|-------|---------|------------------------------------------------|
| $F$  | 15    | 1       | формат команди                                 |
| $OP$ | 14–11 | 4       | код операції                                   |
| $TA$ | 10    | 1       | тип адресації: 0 – пряма, 1 – пряма регістрова |
| $A$  | 9–0   | 10      | адреса ОП або номер регістра/порту             |

| Регістр | Призначення                                                 |
|---------|-------------------------------------------------------------|
| РС      | 7-й біт містить ознаку готовності пристрою до обміну даними |
| РД      | містить дані, що зчитуються із зовнішнього пристрою         |

## Архітектура системи

### Структура НОЗП

| Регістр | Призначення             |
|---------|-------------------------|
| R1      | стан порту              |
| R2      | операнд $X$             |
| R3      | лічильник команд        |
| R4      | поточна команда         |
| R5      | операнд $Y$             |
| R6      | операнд $Z$             |
| R7      | результат $X \& Y$      |
| R8      | множене                 |
| R9      | старші біти результату  |
| R10     | молодші біти результату |
| R11     | лічильник циклів        |

### Структура ПМК

| Адреса | Мікропрограма |
|--------|---------------|
| 0h     | старт         |
| 1h     | test          |
| 2h     | ready_test    |
| 3h     | input         |
| 4h     | load_y        |
| 5h     | load_z        |
| 6h     | op_xy         |
| 7h     | multiply      |
| 8h     | write         |
| 9h     | end           |

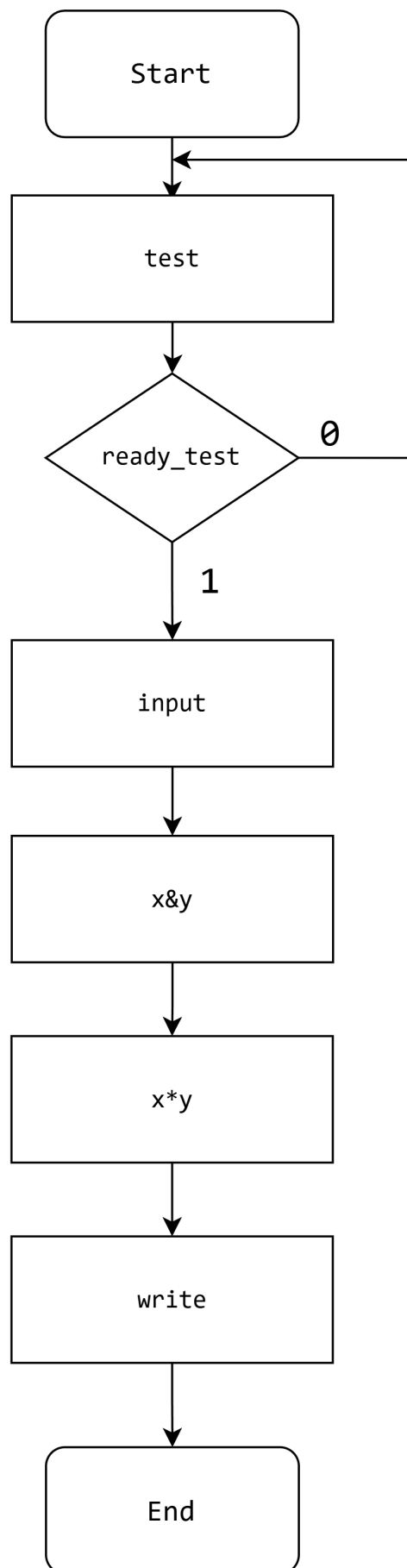
### Структура ОП

| Адреса  | Вміст                    |
|---------|--------------------------|
| 1h      | значення $Y$             |
| 2h      | значення $Z$             |
| 3h      | старше слово результату  |
| 4h      | молодше слово результату |
| 20h–28h | програма                 |

### Структура ЗП

| Адреса | Вміст |
|--------|-------|
| 02h    | PC    |
| 04h    | РД    |

## Змістовний мікроалгоритм



## Система команд

| Команда    | Код програми | Операція                            |
|------------|--------------|-------------------------------------|
| test       | 1h           | тестування готовності порту         |
| ready_test | 2h           | перевірка біта готовності пристрою  |
| input      | 3h           | введення значення $X$ з РД          |
| load_y     | 4h           | зчитування операнда $Y$ з ОП        |
| load_z     | 5h           | зчитування операнда $Z$ з ОП        |
| op_xy      | 6h           | виконання операції $X \& Y$         |
| multiply   | 7h           | множення результату $X \& Y$ на $Z$ |
| write      | 8h           | запис результату множення в ОП      |
| end        | 9h           | завершення мікропрограми            |

## Обрахунки

$$X = 14_{10} = 00.00\ 0000\ 0000\ 1110_2 = 000E_{16},$$

$$Y = 11_{10} = 00.00\ 0000\ 0000\ 1011_2 = 000B_{16},$$

$$Z = 5_{10} = 00.00\ 0000\ 0000\ 0101_2 = 0005_{16},$$

$$X \& Y = 14_{10} \& 11_{10} = 10_{10} = 000A_{16},$$

$$F = (X \& Y) \times Z = 10_{10} \times 5_{10} = 50_{10} = 0032_{16}.$$

## Мікропрограма

```

\=====
\ LAB4
\ F = (X & Y) * Z
\ X -> RD zovnishnoho prystroiui
\ Y, Z -> OP
\ op_xy, multiply -> priama rehistrova adresatsiia
\ multiply -> 3-y sposob
\=====

link l1:ct
link l2:rdm
link l3:rdd
link ewh:10
link M: z,z,z,z,z,z,z,z,14,13,12,11
link ra:3,2,1,0

accept rdm_delay:3

accept dev[0]:I, 02h, 04h, 6, 2
accept dev_buf[0]:000Eh          \ X = 14

accept R1:0000h                  \ stan portu
accept R2:0000h                  \ X
accept R3:0020h                  \ adresa potочноi komandy v OP
accept R4:0003h                  \ adresa persшого слова rezultatu v OP

```

```

accept R5:0000h          \ Y
accept R6:0000h          \ Z
accept R7:0000h          \ X & Y
accept R8:0000h          \ mnozhene (nerukhome)
accept R9:0000h          \ registr komandy
accept R10:0000h         \ starsha chastyna rezultatu
accept R11:0000h         \ molodsha chastyna rezultatu
accept R12:0000h         \ lichylnyk cykliv

\----- dany v OP -----

dw 1h:000Bh              \ Y = 11
dw 2h:0005h              \ Z = 5
dw 3h:0000h              \ result high
dw 4h:0000h              \ result low

\----- prohrama v OP -----

dw 20h:0802h             \ test RS
dw 21h:1002h             \ ready_test RS
dw 22h:1804h             \ input RD
dw 23h:2001h             \ load_y [1h]
dw 24h:2802h             \ load_z [2h]
dw 25h:3405h             \ op_xy R5
dw 26h:3C06h             \ multiply R6
dw 27h:4003h             \ write [3h]
dw 28h:4800h             \ end

\----- vektory komand -----

ORG 0h {cjp nz, start;}
ORG 1h {cjp nz, test;}
ORG 2h {cjp nz, ready_test;}
ORG 3h {cjp nz, input;}
ORG 4h {cjp nz, load_y;}
ORG 5h {cjp nz, load_z;}
ORG 6h {cjp nz, op_xy;}
ORG 7h {cjp nz, multiply;}
ORG 8h {cjp nz, write_res;}
ORG 9h {cjp nz, end;}

\----- makrosy -----

macro next_com: {cjp nz, next;}
macro inc reg: {add reg, z, nz;}
macro op_addr: {or nil, R9, z; oey; ewl;}

\=====
\ Start / fetch
\=====

start
{xor nil, R0, R0; oey; ewh;}          \ ochystyty starshi rozriady adresy

```

```

{cjp nz, read_com;}

read_com
{xor nil, R0, R0; oey; ewh;}
{or nil, R3, z; oey; ewl;}          \ adresa potochnoi komandy
{cjp rdm, cp; R; or R9, bus_d, z;}  \ R9 <- komanda
{cjp nz, check;}

check
{and nil, R9, 8000h; load RM, flags;} \ odnoadresna komanda?
{cjp not RM_z, end;}
{or nil, R9, z; oey; jmap;}          \ perexid na mikoprohramu operatsii

next
{inc R3;}                          \ nastupna komanda
{cjp nz, read_com;}

\=====
\ test RS
\=====

test
{op_addr;}
{cjp rdd, cp; I; or R1, bus_d, z;}   \ R1 <- RS
{next_com;}

\=====
\ ready_test
\=====

ready_test
{and nil, R1, 0080h; load RM, flags;} \ bit gotovnosti
{cjp not RM_z, ready_ok;}
{op_addr;}
{cjp rdd, cp; I; or R1, bus_d, z;}   \ povtorno chytaiemo RS
{cjp nz, ready_test;}

ready_ok
{next_com;}

\=====
\ input RD -> R2
\=====

input
{op_addr;}
{cjp rdd, cp; I; or R2, bus_d, z;}   \ R2 <- X
{next_com;}

\=====
\ load_y
\=====

```

```

load_y
{op_addr;}
{cjp rdm, cp; R; or R5, bus_d, z;}    \ R5 <- Y
{next_com;}

\=====
\ load_z
\=====

load_z
{op_addr;}
{cjp rdm, cp; R; or R6, bus_d, z;}    \ R6 <- Z
{next_com;}

\=====
\ op_xy : R7 := R2 AND R[A]
\ dla komandy 3405h A=5 -> R5
\=====

op_xy
{or nil, R9, z; oey; ewa;}           \ zawayazhyty nomer rehistra v RA
{xor R7, R7;}
{or R7, R2, z;}
{and R7, ra;}
{next_com;}

\=====
\ multiply : 3-y sposob
\ multiplicand = R7 -> R8 (nerukhome)
\ multiplier   = R[A] -> R11
\ result pair  = R10:R11
\=====

multiply
{or nil, R9, z; oey; ewa;}           \ A -> RA
{or R8, R7, z;}                      \ R8 := X & Y
{xor R10, R10;}                      \ high := 0
{xor R11, R11;}
{or R11, ra;}                        \ low := Z
{or R12, 0010h, z;}                 \ CT := 16

M1
{and nil, R11, 0001h; load RM, flags;} \ test low bit
{cjp RM_z, M2;}
{add R10, R10, R8, z;}               \ high := high + multiplicand

M2
{or srl, R10, z;}                    \ high >>= 1
{or sr.9, R11, z;}                   \ low >>= 1 with carry from high
{sub R12, R12, z, z; load RM, flags; cem_c;}
{cjp not RM_z, M1;}
{next_com;}

```

```

\=====
\ write result to OP
\=====

write_res
{xor nil, R0, R0; oey; ewh;}
{or nil, R4, z; oey; ewl;} \ adresa 3h
{or nil, R10, z; oey; cjp rdm, cp; W;} \ M[3h] <- high
{inc R4;}
{xor nil, R0, R0; oey; ewh;}
{or nil, R4, z; oey; ewl;} \ adresa 4h
{or nil, R11, z; oey; cjp rdm, cp; W;} \ M[4h] <- low
{next_com;}

\=====
\ end
\=====

end
{}

```

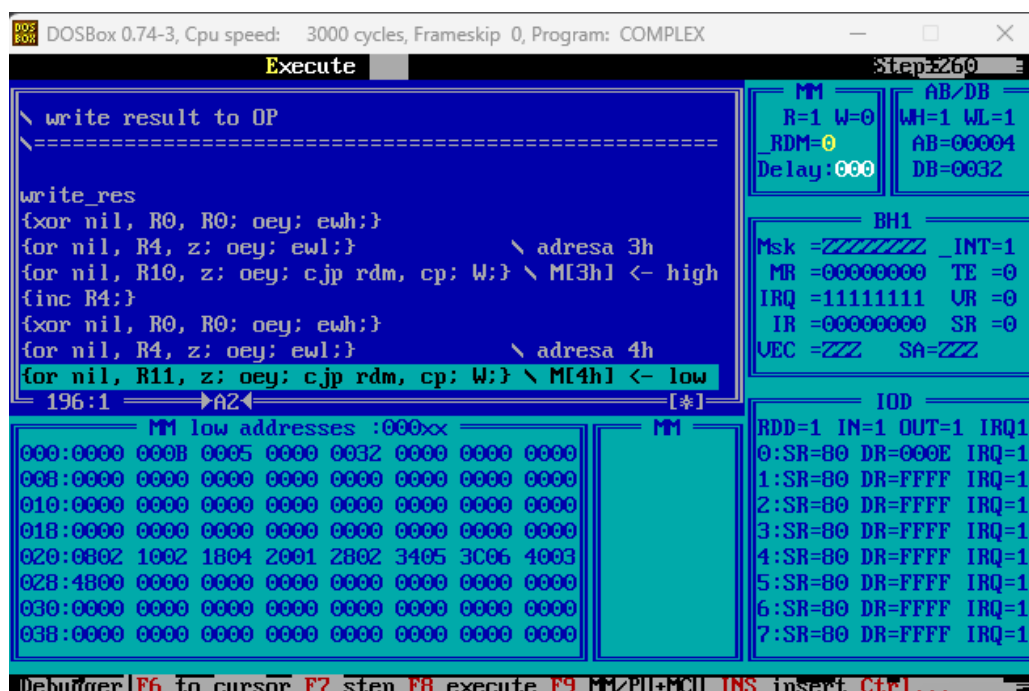
## Результат виконання

$$F = (X \& Y) \times Z = 50_{10} = 00.00\ 0000\ 0000\ 0000\ 0000\ 0000\ 0011\ 0010_2 = 0000\ 0032_{16}$$

Після виконання мікропрограми в оперативній пам'яті мають бути записані:

| Адреса | Значення |
|--------|----------|
| M[3h]  | 0000h    |
| M[4h]  | 0032h    |

І так, запустимо DosBox:



Як бачимо, значення співпали.

## Висновок

У результаті виконання лабораторної роботи було сформовано структуру системи, визначено варіант завдання та побудовано змістовний мікроалгоритм обробки даних. Для мого варіанта реалізовано операцію  $F = (X \& Y) \times Z$ , де операнд  $X$  надходить із зовнішнього пристрою, операнди  $Y$  і  $Z$  зчитуються з оперативної пам'яті, а результат записується назад в ОП. Було складено систему команд, виконано ручні обрахунки та підготовлено мікропрограму, яка відображає етапи тестування пристрою, вводу даних, обчислення логічної операції, множення та запису результату.

## Відповіді на контрольні питання

1. **Охарактеризуйте етапи виконання команд основної групи, передачі керування, введення-виведення, системних.**

Команди основної групи виконуються через вибірку команди, дешифрування, вибірку операндів, виконання операції та запис результату. Для команд передачі керування додатково аналізується умова переходу та змінюється лічильник команд. Для команд введення-виведення виконується обмін із портами пристроїв та перевірка сигналів готовності. Системні команди змінюють режими роботи процесора, стан переривань або службові параметри системи.

2. **Охарактеризуйте основні способи адресації операндів з використанням і без використання РЗП.**

Без використання РЗП застосовують пряму, непряму та безпосередню адресації. Пряма адресація задає точну адресу операнда, непряма використовує адресу вказівника, а безпосередня містить сам операнд у полі команди. З використанням РЗП реалізують пряму регістрову, базову та індексну адресації, де виконавча адреса або сам операнд визначаються через регістри загального призначення.

3. **Як забезпечити правильне зчитування і запис даних у пам'ять з урахуванням швидкодії пам'яті?**

Для цього використовують сигнали готовності пам'яті та цикли очікування. Після подачі сигналу читання або запису процесор повинен дочекатися сигналу RDM. Якщо доступ повільний, у мікропрограмі вводяться додаткові такти очікування, які узгоджують роботу пам'яті та процесора.

4. **Яким чином можна керувати записом інформації в RA і RB, навіщо використовуються зазначені регістри?**

Запис у RA і RB виконується спеціальними керуючими сигналами, які подаються з ПМК. Ці регістри використовуються для вибору операндів у НОЗП та організації доступу до двох джерел даних одночасно. Через них задаються адреси регістрів, які підключаються до входів арифметико-логічного блока.

5. **Поясніть призначення директив мікроасемблера, які визначають роботу з пам'яттю.**

Директива ORG задає адресу розміщення мікрокоманд, DW використовується для ініціалізації комірок пам'яті, ACCEPT задає початковий стан регістрів, пам'яті та зовнішніх пристроїв, а LINK визначає конфігурацію зв'язків між шинами, регістрами та входами ПМК.

6. **Як забезпечити обмін даними між процесором і ЗП в програмному режимі?**

У програмному режимі процесор звертається до регістрів стану і даних зовнішнього пристрою через відповідні адреси портів. Спочатку перевіряється біт готовності в РС, після чого виконується читання або запис через РД. Такий підхід дозволяє уникнути конфлікту між швидкодією процесора і зовнішнього пристрою.